VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Data Structures and Algorithms - CO2004

ASSIGNMENT 3

MUSIC RECOMMENDATION SYSTEM FOR BotKify USING GRAPHS

Version 1.0

Ho Chi Minh City, 04/2026

1 Learning Outcomes

After completing this assignment, students will review and proficiently use several key concepts.

- Design and implement a graph data structure using an adjacency list. This includes operations to add vertices, add edges, and query adjacent vertices.
- Implement the Breadth-First Search algorithm to traverse the graph. This will be used to recommend related songs starting from an initial song.
- Apply the connected components algorithm to cluster songs into independent playlists.
- Implement the Dijkstra algorithm to find the shortest path. This helps achieve the smoothest transition between two songs in a weighted graph.
- Calculate the in-degree of the graph to identify the hub song of the network.
- Apply the Depth-First Search algorithm to detect cycles within a directed graph.
- Simulate auxiliary data structures such as queues, tracing arrays, and recursive stacks without using built-in libraries.

2 Introduction

In this final assignment, students will build BotKify. This is a music recommendation system utilizing a graph data structure. The system models the relationships between songs as a weighted graph. Each vertex represents a song. Each edge represents the level of similarity between two songs.

Students will implement classic graph algorithms on this foundation. These algorithms include Breadth-First Search, finding connected components, and the Dijkstra algorithm for the shortest path. These algorithms will support features such as recommending related songs, automatically clustering playlists, and finding the smoothest music transition path.

3 Description

3.1 Graph Data Structure

3.1.1 Edge

This `struct` represents a directed edge in the graph. It contains the `target` variable to store the identifier of the destination vertex. It also contains the `weight` variable to store the weight

of the edge. This weight indicates the cost or deviation during a music transition.

- `T target`: The identifier of the destination vertex pointed to by this edge. It uses the generic type `T`.
- `double weight`: The weight of the edge. It represents the cost or similarity level between two songs.

3.1.2 AdjacencyNode

This `struct` acts as an adjacency node. It includes `vertex` as the current vertex. It also includes a dynamic array `vector<Edge> neighbors` containing all edges originating from that vertex.

- `T vertex`: The identifier value of the current vertex.
- `vector<Edge> neighbors`: The list of edges originating from this vertex. Each element contains the destination vertex information and weight.

3.1.3 Graph Class

The generic template class `Graph<T>` manages the entire network graph. This class uses `vector<AdjacencyNode> adjList` as an adjacency list. It builds linear search functions such as `getVertexIndex` to manage adding vertices via `addVertex` and adding edges via `addEdge`.

1. Protected attributes:
   - `vector<AdjacencyNode> adjList`: The adjacency list storing all vertices and their corresponding adjacent edges in the graph.
2. Protected methods:
   - `int getVertexIndex(const T& vertex) const`: Performs a linear search through `adjList` to return the index of the `vertex`. It returns -1 if the vertex does not exist.
3. Public methods:
   - `void addVertex(const T& vertex)`: Adds a new vertex to the graph. It checks using `getVertexIndex` to avoid duplicates. If the vertex does not exist, it creates a new `AdjacencyNode` and pushes it into `adjList`.
   - `void addEdge(const T& source, const T& target, double weight, bool isBidirectional = false)`: Adds a weighted edge from `source` to `target`. It automatically calls `addVertex` for both vertices. If `isBidirectional` is true, it adds a reverse edge from `target` to `source`.

- `vector<Edge> getNeighbors(const T& vertex) const`: Returns the list of adjacent edges for the `vertex`. It returns an empty vector if the vertex does not exist.
- `bool hasVertex(const T& vertex) const`: Checks whether the `vertex` exists in the graph. It returns true or false.

3.1.4 Graph Structure Example

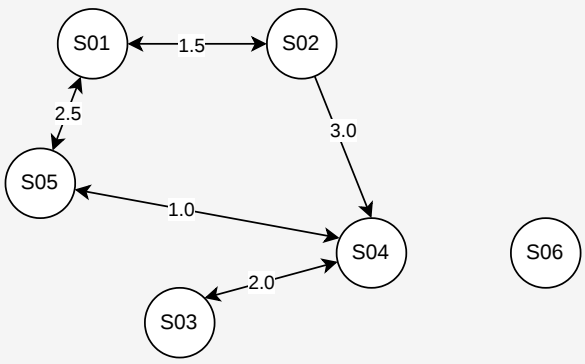
Illustration Example: Graph Structure

Suppose we build a simple graph consisting of the following 6 songs.

ID	Song Name	Artist	Genre
S01	Chung Ta Cua Tuong Lai	Son Tung M-TP	Pop
S02	Muon Roi Ma Sao Con	Son Tung M-TP	Pop
S03	Don't Coi	RPT Orijinn	Rap
S04	Thuy Trieu	Quang Hung MasterD	Pop-Dance
S05	Waiting For You	MONO	Pop
S06	Co Don Tren Sofa	Ho Ngoc Ha	Ballad

The connecting edges are defined as follows.

- `S01` $\leftrightarrow$ `S02` with a weight of 1.5 as bidirectional
- `S02` $\rightarrow$ `S04` with a weight of 3.0 as unidirectional
- `S04` $\leftrightarrow$ `S03` with a weight of 2.0 as bidirectional
- `S01` $\leftrightarrow$ `S05` with a weight of 2.5 as bidirectional
- `S05` $\leftrightarrow$ `S04` with a weight of 1.0 as bidirectional
- `S06` has no connections to any song and forms an isolated vertex



Hình 1: Graph Example

3.2 Music Graph

3.2.1 Introduction

The music graph in BotKify is a directed and weighted graph. Each song is a vertex of type `string`. The relationship between two songs is an edge. Detailed song data is defined through the `struct Song` which includes the following fields.

- `string id`: The unique identifier of the song, for example `"S01"`.
- `string title`: The name of the song.
- `string artist`: The name of the performing artist.
- `string genre`: The music genre of the song.

Additionally, the `struct SongEntry` is declared as private inside the `MusicGraph` class. It encapsulates a pair containing the following information.

- `string id`: The identifier, used as a quick lookup key.
- `Song data`: The `Song` object containing all detailed information of the song.

3.2.2 MusicGraph Class

The `MusicGraph` class inherits directly from `Graph<string>`. It encapsulates the `vector<SongEntry> songsList` array to store the entity data of the songs. This separates the data from the adjacency graph structure. The class is responsible for providing interfaces for business operations.

1. Private attributes:
   - `vector<SongEntry> songsList`: The array storing the list of song entities. This is separated from the adjacency graph structure.
2. Private methods:
   - `int getSongIndex(const string& id) const`: Performs a linear search in `songsList` to return the index of the song with the given `id`. It returns -1 if not found.
   - `bool isVisited(const string& id, const vector<string>& visitedList) const`: Checks if a vertex has been visited by performing a linear search in the `visitedList`. This function is reused by the BFS and connected components algorithms.

3. Public methods:
   - `void addSong(const string& id, const string& title, const string& artist, const string& genre)`: Adds a new song to the system. It checks for duplicates using `getSongIndex`. Then it creates `Song` and `SongEntry` objects and pushes them into `songsList`. It simultaneously calls `addVertex(id)` to add the corresponding vertex to the graph.
   - `void printSongInfo(const string& id) const`: Prints song information to the console in the format `[id] title - artist`. This is a utility function used by other methods to display results.
   - `void recommendRelatedSongs(const string& startId) const`: Recommends related songs. See details in section 3.3.1.
   - `void generatePlaylistsByClusters() const`: Generates playlists by clusters. See details in section 3.3.2.
   - `void findSmoothTransition(const string& startId, const string& endId) const`: Transitions between songs. See details in section 3.3.3.
   - `void findMostPopularSong() const`: Finds the network hub song. See details in section 3.3.4.
   - `void detectMusicLoop() const`: Detects music loops. See details in section 3.3.5.

3.3 System Algorithms

3.3.1 Recommend Related Songs

This is implemented based on the Breadth-First Search algorithm. The system simulates a queue using a `vector<string> customQueue` combined with a `head` pointer. This is used to traverse adjacent songs and provide recommendations.

1. Method: `void recommendRelatedSongs(const string& startId) const`
2. Main local variables:
   - `vector<string> visited`: The list of visited vertices to help avoid infinite loops.
   - `vector<string> customQueue`: Simulates a queue containing vertices waiting to be processed.
   - `int head`: The pointer to the front of the queue. It increments when extracting an element instead of deleting it directly.
3. Operational process:

- Check if the starting vertex exists using `hasVertex(startId)`.
- Add the starting vertex to `visited` and `customQueue`.
- Loop until the queue is fully processed. Extract the element at the `head` position. Traverse all `neighbors` via `getNeighbors`.
- For each unvisited adjacent vertex, add it to `visited` and `customQueue`. Simultaneously print the recommendation information using `printSongInfo`.

Illustration Example: BFS

Call `recommendRelatedSongs("S01")` on the sample graph in section 3.1.

1. Initialization: `visited = [S01]`, `customQueue = [S01]`, `head = 0`.
2. Step 1: Get `S01` with head incremented to 1. Neighbors include `S02` with weight 1.5 and `S05` with weight 2.5. Both are unvisited and added.
   `visited = [S01, S02, S05]`, `customQueue = [S01, S02, S05]`.
   Print: `-> [S02] Muon Roi Ma Sao Con - Son Tung M-TP`
   Print: `-> [S05] Waiting For You - MONO`
3. Step 2: Get `S02` with head incremented to 2. Neighbors include `S01` which is visited and `S04` which is unvisited. Add `S04`.
   Print: `-> [S04] Thuy Trieu - Quang Hung MasterD`
4. Step 3: Get `S05` with head incremented to 3. Neighbors include `S01` and `S04` which are both visited. Add nothing.
5. Step 4: Get `S04` with head incremented to 4. Neighbors include `S03` which is unvisited and `S05` which is visited. Add `S03`.
   Print: `-> [S03] Don't Coi - RPT Orijinn`
6. Step 5: Get `S03` with head incremented to 5. Neighbor `S04` is visited. The algorithm terminates.

The recommendation result in BFS order is S02, S05, S04, S03. Song S06 is not recommended because it is not connected to S01.

3.3.2 Create Playlists by Clusters

This applies the connected components problem. The function traverses all unvisited vertices stored in `visited`. It repeatedly calls the BFS algorithm to group all connected songs into the same playlist cluster.

1. Method: `void generatePlaylistsByClusters() const`

2. Main local variables:
   - `vector<string> globalVisited`: A global list tracking all vertices that already belong to a playlist.
   - `int playlistCount`: A counter for the created playlists.
   - `vector<string> customQueue` and `int head`: A local BFS queue for each cluster.
3. Operational process:
   - Traverse each vertex in `adjList` sequentially.
   - If the vertex does not belong to `globalVisited`, create a new playlist and run BFS from that vertex.
   - BFS groups all connected vertices into the same cluster. It marks them in `globalVisited` and prints the information using `printSongInfo`.
   - The result separates unconnected songs into independent playlists.

Illustration Example: Connected Components

Call `generatePlaylistsByClusters()` on the sample graph in section 3.1.

1. Traverse vertex S01. S01 does not belong to `globalVisited` so create Playlist 1. BFS from S01 groups the following: S01, S02, S05, S04, S03. `globalVisited = [S01, S02, S05, S04, S03]`.
2. Traverse vertices S02, S03, S04, S05. All of them already belong to `globalVisited` so skip them.
3. Traverse vertex S06. S06 does not belong to `globalVisited` so create Playlist 2. BFS from S06 only groups S06 because it has no neighbors.

Printed result:

```
=== Playlist 1 ===
  * [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
  * [S02] Muon Roi Ma Sao Con - Son Tung M-TP
  * [S05] Waiting For You - MONO
  * [S04] Thuy Trieu - Quang Hung MasterD
  * [S03] Don't Coi - RPT Orijinn
=== Playlist 2 ===
  * [S06] Co Don Tren Sofa - Ho Ngoc Ha
```

Note that song S06 has no connections so it forms an independent playlist. Songs S01 to S05 are connected to each other so they belong to the same playlist.

3.3.3 Song Transition

This uses the Dijkstra algorithm to find the shortest path for the smoothest transition between two songs. The algorithm applies a traditional array traversal method to find the vertex with the minimum distance. This has a time complexity of $O(V^2)$. It then uses the `prev` array to trace the playback order.

1. Method: `void findSmoothTransition(const string& startId, const string& endId) const`
2. Main local variables:
   - `int n`: The number of vertices in the graph retrieved from `adjList.size()`.
   - `vector<double> dist`: The array of shortest distances from the starting vertex. It is initialized with the value `INF = 999999999.0`.
   - `vector<int> prev`: The tracing array. It stores the index of the previous vertex on the shortest path. It is initialized with -1.
   - `vector<bool> visited`: The array marking processed vertices.
   - `vector<string> path`: The array storing the resulting path after tracing.
3. Operational process:
   - Check if the starting vertex and ending vertex exist using `getVertexIndex`.
   - Initialize `dist[startIdx] = 0` and the remaining vertices to `INF`.
   - Loop V times. Find the unvisited vertex `u` with the smallest `dist` by linear traversal $O(V)$. Mark `visited[u] = true`.
   - Perform distance update. For each adjacent edge of `u`, check the distance. If `dist[u] + weight < dist[v]`, update `dist[v]` and assign `prev[v] = u`.
   - Trace back. From the destination vertex, traverse the `prev` array backward to reconstruct the path. Print the playback order using `printSongInfo`.

Illustration Example: Dijkstra

Call `findSmoothTransition("S01", "S03")` on the sample graph in section 3.1.
Dijkstra state table with vertices in the order of adjList: S01, S02, S03, S04, S05, S06.

Step	dist[S01]	dist[S02]	dist[S03]	dist[S04]	dist[S05]	dist[S06]	Select u
Init	0	∞	∞	∞	∞	∞	—
1	0	1.5	∞	∞	2.5	∞	S01
2	0	1.5	∞	4.5	2.5	∞	S02
3	0	1.5	∞	3.5	2.5	∞	S05
4	0	1.5	5.5	3.5	2.5	∞	S04

Calculation process details:

- Step 1 with u = S01: Calculate `dist[S02] = 0 + 1.5 = 1.5` and `dist[S05] = 0 + 2.5 = 2.5`. Assign `prev[S02] = S01` and `prev[S05] = S01`.
- Step 2 with u = S02: S01 is visited. Calculate `dist[S04] = 1.5 + 3.0 = 4.5`. Assign `prev[S04] = S02`.
- Step 3 with u = S05: S01 is visited. The new distance to `S04` is $2.5 + 1.0 = 3.5 < 4.5$. Therefore, update `dist[S04] = 3.5` and `prev[S04] = S05`.
- Step 4 with u = S04: This is the adjacent vertex to the destination S03. Calculate `dist[S03] = 3.5 + 2.0 = 5.5`. Assign `prev[S03] = S04`. The algorithm terminates because `u == endIdx`.

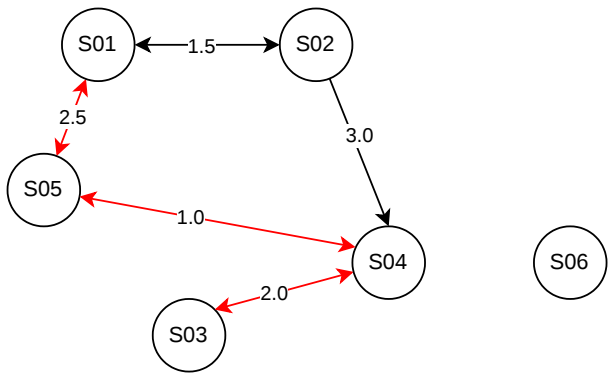
Trace back the `prev` array from S03:

```
S03 <- prev[S03]=S04 <- prev[S04]=S05 <- prev[S05]=S01 (start)
```

Printed result:

```
-> Total Deviation (Cost): 5.5
-> Playback Order:
   1. [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
   2. [S05] Waiting For You - MONO
   3. [S04] Thuy Trieu - Quang Hung MasterD
   4. [S03] Don't Coi - RPT Orijinn
```

Note that the shortest path from S01 to S03 goes through S05 and then to S04 with a total cost of 5.5. This is better than going through the S02 branch which has a total cost of 6.5.



Hình 2: Music graph after finding the shortest path

3.3.4 Find Network Hub Song

This identifies the national song by calculating the in-degree of the graph. The system traverses the entire adjacency list to count the number of times a vertex appears as the `target` destination. The vertex with the highest in-degree is the popular song. It can easily be selected to play next from many other songs in the system.

1. Method: `void findMostPopularSong() const`
2. Main local variables:
   - `int n`: The number of vertices in the graph retrieved from `adjList.size()`.
   - `vector<int> inDegree`: The array storing the in-degree of each vertex. It is initialized with 0.
   - `int maxIdx`: The index of the vertex with the highest in-degree.
3. Operational process:
   - Initialize the `inDegree` array of size n with the value 0.
   - Traverse each vertex i in `adjList`. For each adjacent `Edge` of vertex i, lookup the index of `edge.target` using `getVertexIndex`. Increment `inDegree` at that index by 1.
   - Find the vertex with the highest `inDegree` by linear traversal.
   - Print the network hub song information using `printSongInfo` along with the in-degree value.

Illustration Example: In-degree

Call `findMostPopularSong()` on the sample graph in section 3.1.
Recall the directed edges in the graph:

- S01 → S02 with a weight of 1.5 and vice versa S02 → S01
- S02 → S04 with a weight of 3.0. This is a unidirectional type.
- S04 → S03 with a weight of 2.0 and vice versa S03 → S04
- S01 → S05 with a weight of 2.5 and vice versa S05 → S01
- S05 → S04 with a weight of 1.0 and vice versa S04 → S05

Count the in-degree based on the number of edges coming to each vertex:

Vertex	Incoming edges	In-degree
S01	from S02, from S05	2
S02	from S01	1
S03	from S04	1
S04	from S02, from S03, from S05	3
S05	from S01, from S04	2
S06	none	0

Printed result:

```
-> Network Hub Song:
   [S04] Thuy Trieu - Quang Hung MasterD
   (In-degree: 3)
```

Note that song S04 Thuy Trieu has the highest in-degree of 3. This means there are 3 other songs that can transition to S04. This is the national song that can most easily be played next from multiple other sources in the system. Song S06 has an in-degree of 0 because it is an isolated vertex.

3.3.5 Detect Music Loop

This applies the Depth-First Search algorithm on a directed graph to detect cycles. The system uses a dynamic array to simulate and track the states of vertices currently in the recursive branch via the `recursionStack` array. If the traversal process encounters a vertex already in this tracking list, an endless music loop is detected and extracted.

1. Method: `void detectMusicLoop() const`
2. Main variables and helper functions:
   - `vector<bool> visited`: The array marking the visited vertices.
   - `vector<bool> recursionStack`: The array tracking the vertices currently on the recursive branch. A vertex is marked true when the traversal begins and false when backtracking.
   - `vector<int> parent`: The tracing array. It stores the index of the parent vertex on the DFS path. It is used to extract the cycle when detected.
   - Recursive function `dfsCycleHelper(int idx, ...)`: Executes DFS from vertex `idx`. It marks `visited[idx] = true` and `recursionStack[idx] = true`. For each adjacent vertex `v`:
     - If `v` is unvisited: record `parent[v] = idx` and call `dfsCycleHelper` recursively.
     - If `v` is already in the `recursionStack`: a cycle is detected. Trace backward through `parent` from `idx` to `v` to extract the list of vertices forming the loop.

     When backtracking, the algorithm resets `recursionStack[idx] = false`.
3. Operational process:
   - Initialize `visited` and `recursionStack` arrays of size n to false. Initialize the `parent` array to -1.
   - Traverse each vertex sequentially. If the vertex is unvisited, call `dfsCycleHelper` from that vertex.
   - When the first cycle is detected, extract the loop. Print the information of the songs in the cycle using `printSongInfo` and then terminate.
   - If all vertices are traversed without detecting a cycle, print a message indicating no loop was found.

Illustration Example: Cycle Detection

Call `detectMusicLoop()` on the sample graph in section 3.1.
Initialization: The `visited` and `recursionStack` arrays are both false. The `parent` array is filled with -1.

1. DFS from S01: Mark `visited[S01] = true` and `recStack[S01] = true`.
   Traverse the first neighbor, which is `S02`.
2. DFS from S02: Mark `visited[S02] = true`, `recStack[S02] = true`, and `parent[S02] = S01`.
   Traverse the first neighbor of S02, which is `S01`.

Check and find that `S01` is already in the `recursionStack`. The system detects a cycle.

3. Extract the cycle: Trace backward from S02 through `parent`.
   S02 points back to S01. S01 is the vertex causing the cycle, so stop tracing.
   The resulting loop is: S01 → S02 → S01.

Printed result:

```
-> Music loop detected!
-> Loop:
   [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
   [S02] Muon Roi Ma Sao Con - Son Tung M-TP
   [S01] Chung Ta Cua Tuong Lai - Son Tung M-TP
```

Note that the cycle from S01 to S02 and back to S01 exists due to the bidirectional edge between these two songs. The algorithm is designed to stop immediately upon detecting the first cycle.

4 Requirements

To complete this assignment, students must accomplish the following steps.

1. Read this entire description file.
2. Download the initial.zip file and extract it. After extracting, students will receive the files main.cpp, main.h, MusicGraph.h, MusicGraph.cpp, and sample reading data files. Students must submit exactly 2 files: MusicGraph.h and MusicGraph.cpp. Therefore, they are not allowed to modify the main.h file when testing the program.
3. Students use the following command to compile:
   g++ -o main main.cpp MusicGraph.cpp -I . -std=c++11
   The above command is used in the command prompt or terminal to compile and run the program. If students use an IDE to run the program, they need to pay attention. They should add all files to the project workspace of the IDE. They should adjust the IDE compile command accordingly. IDEs usually provide buttons for Building and Running the program. When pressing Build, the IDE will execute a corresponding compile command. Typically, this command only compiles the main.cpp file. Students need to figure out how to configure the IDE to change the compile command. They must add the MusicGraph.cpp

file and add the options -std=c++11 and -I .

4. The program will be graded on a Unix platform. The grading platform and the student's compiler might differ from the actual grading environment. The submission environment on BKeL is set up to mimic the actual grading environment. Students must test their program on the submission environment. They must fix all errors occurring on the BKeL submission environment to get the correct results during actual grading.
5. Modify the files MusicGraph.h and MusicGraph.cpp to complete this assignment and ensure the following two requirements:
   - There is only one include statement in the `MusicGraph.h` file:

     #include "main.h"

   - In the `MusicGraph.cpp` file, there is only one include statement:

     #include "MusicGraph.h"

   - Apart from the two includes above, no other include statements are allowed in these files.
   - Implement the functions described in the tasks of this assignment.
6. Students are encouraged to write additional functions to complete this assignment.

5 Submission

Students only submit 2 files: MusicGraph.h and MusicGraph.cpp. They must do this before the deadline provided in the "Assignment 3 - Submission" link. A few simple test cases are used to check the students' work to ensure their code compiles and runs. Students can submit as many times as they want. However, only the final submission is graded.

The system cannot handle the load if too many students submit at the same time. Therefore, students should submit as early as possible. Students assume their own risk if submitting close to the deadline. This includes the period within 1 hour before the deadline. When the deadline passes, the system will close. Students will no longer be able to submit. Submissions via other methods are not accepted.

6 Other Regulations

1. Students must complete this assignment themselves. They must prevent others from stealing their results. Otherwise, students will be handled according to university regulations

for cheating.

2. All decisions made by the instructor in charge of the assignment are final.
3. Students will not be provided with test cases after grading.
4. The content of the assignment will be harmonized with some questions in the Final Exam with similar content.
5. If students use source code from AI programs or automatic code generation tools without clearly understanding how it works and its meaning, the assignment will be graded 0 points.

7 Harmony for Assignment

The final exam of the course will contain some Harmony questions related to the content of the assignment.

Students must solve the assignment using their own abilities. If students cheat in the assignment, they will not be able to answer the Harmony questions. They will receive 0 points for the assignment.

Students must pay attention to answering the Harmony questions in the final exam. Cases of not answering will result in 0 points for the assignment. This also leads to failing the course. Explanations are not accepted, and there are no exceptions.

8 Plagiarism

The assignment must be done by the student themselves. Students will be considered to have cheated if any of the following occur:

- There is abnormal similarity between the source codes of submissions. In this case, ALL submissions are considered plagiarism. Therefore, students must protect their assignment source code.
- A student's work is submitted by another student.
- Students do not understand the source code they wrote themselves. This excludes the code provided in the starter template. Students can consult any resource. However, they must ensure they fully understand the meaning of all lines of code they write. If they do not clearly understand the source code from their reference, they are strictly warned NOT to use that source code. Instead, they should use what they have learned to write the program.
- Mistakenly submitting another student's work on their personal account.

- Using source code from tools capable of generating code without understanding its meaning.

In case of a cheating conclusion, the student will receive 0 points for the entire course. This applies not just to the assignment.

EXPLANATIONS ARE NOT ACCEPTED, AND THERE ARE NO EXCEPTIONS!

After each assignment submission, some students will be randomly called for an interview. This is to prove that the submitted assignment was done by themselves.

——————————**HẾT**——————————